

UMBRA SPACE TECH ASSESSMENT

Asynchronous Propulsion Server - Problem Statement

You are a member of the Flight Software Team at Umbra Space and are responsible for writing code that manages the satellite’s propulsion system. Firing the propulsion system involves waiting for a certain period of time before ignition. The following is an example usage of this system:

- At absolute time $t = 0$, send a command to the computer to fire the propulsion in 15 seconds
 - At absolute time $t = 2$, send a command to the computer to fire the propulsion in 30 seconds
 - At absolute time $t = 32$, the computer begins firing the propulsion
-

General Requirements

- Commands should be delineated by newlines.
- The Flight computer should be able to fire the thruster multiple times in a single execution of the program.

Input Interfaces

You may use exactly one of the following interfaces for getting data into and out of your program, whichever is more convenient.

Standard Input/Standard Output

- If your language of choice has good Standard Input/Standard Output support, *please use it*.

TCP

- Some languages make asynchronous IO with Standard Input/Output cumbersome, so we have included TCP.
 - A sample TCP client, `propulsion_tcp_client.py`, is provided that plumbs standard input to TCP writes and likewise plumbs TCP reads to standard output.
 - If you do choose to use TCP, please have your server listen on port `8124`.
-

Expected Behaviors

Condition	Behavior
Flight computer receives a command with a relative time to fire propulsion	Start a countdown; when time elapses, print “firing now!”
Another command arrives before the current one fires	New command overwrites the previous pending command
Time of -1 entered	All outstanding commands to fire the propulsion are cancelled

Sample Client Execution

```
./your_program  
15  
30  
firing now!
```

Submitting your code

Notes for Developers

You may use any programming language. Whilst we prefer async Rust, please use the language you are strongest in, as the technical interview will dig deeply into your submission. We pray it only takes you an hour or so, plus a bit to polish it such that it will be easy to review.

We appreciate concise, thoughtfully scoped submissions. This exercise can be solved elegantly with less than 100 lines (plus comments and tests). Thank you in advance for avoiding overly complex or multi-thousand-line implementations.

Create a Google Drive folder named `your_name_technical_assessment` containing your program's source code, a `README` describing your approach, assumptions, and build/run instructions, as well as any required libraries or installation steps, ensuring no build artifacts are included. Keep the code clear and readable, and submit whenever you're ready, as there is no deadline.

Post-Submission

Reviews are typically completed within 1 business day. If the submission looks good, a 1-hour technical interview will be scheduled ASAP. So please feel free to assume your submission makes the cut (roughly 2 in 3 submissions do), and submit, with your assessment, three 1-hour options over the following couple of business days for the technical interview.